

Doctor Platform

An AI-first, **multi-tenant** doctor **listing** · **lead-capture** · **booking** marketplace. Research, product spec, role model, multi-agent AI design, architecture, deployment & cost, the 8-agent **Engineering Operating System**, delivery workflow, and the full governance rulebook.

 Inspired by Practo & Apollo247

 Udaipur → India

 Vercel + Supabase

 Multi-tenant (clinic = tenant)

 8 build agents · 5 product agents

Prepared for: Naman · **Market:** Udaipur, Rajasthan (→ India)

Date: 16 June 2026 · **Status:** Blueprint for build

Table of Contents

Ch 1.	Executive Summary
Ch 2.	Market & Competitor Research — Practo & Apollo247
Ch 3.	Product Vision & Scope
Ch 4.	User Roles & RBAC Model
Ch 5.	Core Feature Modules
Ch 6.	The AI Layer — Multiple Agents & LLM Choice
Ch 7.	System Architecture
Ch 8.	Data Model
Ch 9.	Frontend Specification
Ch 10.	Backend & API Specification
Ch 11.	Deployment — Vercel + Supabase
Ch 12.	Cost Analysis (INR + USD)
Ch 13.	Security & Compliance (India / DPDP)
Ch 14.	Build Methodology — Claude Code + Codex + MCP
Ch 15.	Engineering Operating System — The 8-Agent Roster
Ch 16.	Delivery Workflow & Traceability
Ch 17.	Documentation System & Skills
Ch 18.	Governance Rulebook (full)
Ch 19.	Phased Project Plan
Ch 20.	Appendices — Ready-to-paste Templates

What's new in v1 (Enterprise Governance Edition): multi-tenancy (clinic = tenant) woven through roles, data & security · the **8-agent Engineering Operating System** (Ch. 15) · the **delivery workflow & traceability** model (Ch. 16) · the **documentation system** (Ch. 17) · the full **governance rulebook** (Ch. 18). Product scope is unchanged: listing + leads + booking. Previous version archived at `.versions/v0`.

How to read this: Ch. 1–13 = *product & architecture*. Ch. 14–18 = *how we build & govern it* (agents, workflow, docs, rules). Ch. 20 = copy-paste templates. Working copies of every doc live as markdown in `/docs` (the rulebook mirrors to `/docs/GOVERNANCE.md`).

1. Executive Summary

We are building a focused, **AI-first doctor portal** for **Udaipur, Rajasthan**, designed to expand across India. Patients discover doctors, and the moment they want to connect, a **popup form** captures them as a **lead**; staff then convert that lead into a confirmed **booking**. It takes the most valuable, lowest-cost parts of Practo and Apollo247 — discovery, listings, lead capture, booking — and ships them fast, leaving heavier features (video consult, medicine, labs, payments) for later phases.

WHAT V1 IS

- Doctor/clinic **listings** & onboarding
- Patient **discovery & AI smart search**
- **Lead capture** popup (the "call/book" flow)
- **Appointment booking** + reminders
- **Role-based** admin / manager / doctor / front-desk
- **5 AI agents** assisting throughout

WHAT V1 IS NOT (YET)

- Video / audio / chat consultation
- e-Prescriptions & online payments
- Medicine delivery & lab tests
- Full health-record vault, ABDM/ABHA

Architected so each can bolt on without a rewrite.

Why this approach wins

- **Cheap to launch:** Vercel + Supabase run near-free at MVP volumes (see Ch. 12).
- **Fast to build:** a two-agent workflow (Claude Code + Codex) on a proven stack.
- **Differentiated:** AI search, lead assistance, triage and a staff copilot from day one.
- **Local first:** winning Udaipur's supply (doctors) and demand (patients) before scaling.

The 3-phase roadmap

Phase	Theme	Headline scope
1 — MVP	Listing · Lead · Booking	Listings, search, lead popup, booking, RBAC, 2 AI agents (Smart Search + Lead-Form Assistant). Udaipur launch.
2 — Engage	Convert & retain	Notifications, reviews, full agent suite (triage, staff copilot, content/SEO), analytics, more cities.
3 — Expand	Healthcare super-app	Payments, teleconsult, labs/medicine, health records, ABDM/ABHA.

Recommended LLM stack (answer to "which AI"): serve all models through **Vercel AI Gateway** and route by task — **Claude Haiku 4.5** for cheap high-volume work, **Claude Sonnet 4.6** as the reasoning default, **Claude Opus 4.8** only for the hardest agentic flows, plus a small embedding model in **Supabase pgvector** for semantic search. Full detail in Chapter 6.

2. Market & Competitor Research

Both reference platforms are Indian health-tech leaders. We study them to copy what is proven and cheap, and defer what is heavy and capital-intensive.

PRACTO • Doctor/clinic discovery by location, specialty, availability, reviews • Appointment booking (online + clinic-side) • Practo Ray — clinic management for ~50k doctors: scheduler, EMR/EHR, billing, SMS alerts, reminders, reports • Online video consultation across 25+ specialties; Practo Plus subscription • Medicine ordering & diagnostic tests; ratings/reviews Takeaway: their discovery + booking + clinic-side tools are exactly our v1. Their consult/pharmacy layers are our later phases.	APOLLO247 • Online consultation (video/audio/chat) — 2200+ doctors, 55+ specialties • Medicine delivery (19-min in metros), 75k+ SKUs • Home lab tests via Apollo Diagnostics • Ask Apollo — an AI-backed symptom checker • Health-record vault; Apollo Circle membership Takeaway: Ask Apollo validates our AI symptom-triage agent. The rest is multi-vertical scale we don't need on day one.
---	---

Feature decision matrix

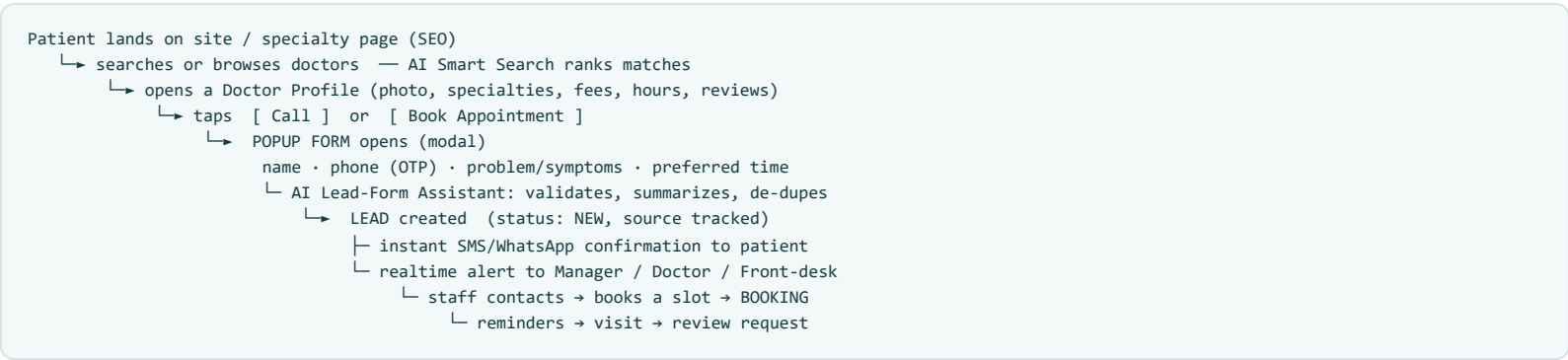
Capability	Practo	Apollo247	Our v1
Doctor discovery & search	✓	✓	Adopt
Doctor listing / profile	✓	✓	Adopt
Lead capture form ("call/book")	partial	partial	Adopt (core)
Appointment booking	✓	✓	Adopt
SMS / WhatsApp reminders	✓	✓	Adopt
Reviews & ratings	✓	✓	Adopt (P2)
AI symptom checker	—	✓ (Ask Apollo)	Adopt (P2)
AI smart search / match	partial	partial	Adopt (differentiator)
Clinic management (EMR/billing)	✓ (Ray)	✓	Defer
Video / audio consultation	✓	✓	Defer (P3)
Online payments	✓	✓	Defer (P3)
Medicine delivery	✓	✓	Skip (P3+)
Lab tests	✓	✓	Skip (P3+)
Health-record vault	✓	✓	Defer

Our edge: we are smaller and local, so we win on **speed of response** (SLA on every lead), **hyperlocal supply** in Udaipur, and **AI woven through the whole funnel** — not bolted on.

3. Product Vision & Scope

Vision: the fastest way for a patient in Udaipur to find the right doctor and get a confirmed appointment — and the easiest way for a doctor to receive qualified, well-summarized patients — with AI removing friction at every step.

The core journey (the "call → popup form" flow, step by step)



In scope (v1)

Patient side	Doctor / staff side
• Search & filters + AI smart search • Doctor profiles & reviews • Lead popup (OTP-verified) • Booking & self-service status • Symptom triage assistant (P2)	• Onboarding, listing, verification/KYC • Availability & slots • Lead pipeline + assignment + SLA • Booking calendar • Dashboards, reports, staff copilot

Explicitly out of scope (deferred)

Video/audio/chat consult · e-prescriptions · online payments · medicine delivery · lab tests · full EHR/health-record vault · ABDM/ABHA · multi-city operations tooling. Each has a reserved hook in the data model and architecture so it can be added without a rewrite.

Guiding principles

- **Speed of contact** is the product — every lead has an SLA timer.
- **Mobile-first, low-end Android friendly**, Hindi + English ready.
- **SEO-first** public pages (ISR) — organic discovery is the cheapest growth.
- **Trust:** verified doctors, moderated reviews, clear consent, medical disclaimers.

4. User Roles & RBAC Model

Authorization is enforced **server-side** with Supabase Auth + **Row Level Security (RLS)** on every table. The browser is never trusted. There are six human roles plus one AI service role.

Role	Who	Can do (summary)
Super Admin	Platform owner	Everything: billing, global settings, create/suspend Admins, read audit log.
Admin	Ops lead	Verify doctors & listings, manage Managers, all leads/bookings, reports, moderate reviews.
Manager	Territory/desk lead	Assigned doctors/clinics + their leads/bookings; assign & follow up leads.
Doctor / Clinic owner	Supply	Own listing, availability, own leads & bookings; manage own front-desk staff.
Front-desk / Receptionist	Clinic staff	That clinic's leads & bookings only.
Patient	Demand	Browse, search, submit lead, book, view own bookings & submit reviews. Guest or account.
AI service role	Agents	Scoped backend identity; can only see what its caller's role can see; every action audit-logged.

Permission matrix

✓ = full · 🕒 = own/assigned scope only · ✕ = none

Action / Role	Super	Admin	Mgr	Doctor	Front-desk	Patient
Platform settings & billing	✓	✕	✕	✕	✕	✕
Create/suspend Admins & Managers	✓	🕒	✕	✕	✕	✕
Verify / approve listings	✓	✓	✕	✕	✕	✕
Create / edit doctor listing	✓	✓	🕒	🕒	✕	✕
View leads	✓	✓	🕒	🕒	🕒	🕒
Assign / reassign leads	✓	✓	🕒	✕	✕	✕
Update lead status / notes	✓	✓	🕒	🕒	🕒	✕
Manage availability / slots	✓	✓	🕒	🕒	🕒	✕
Create / modify booking	✓	✓	🕒	🕒	🕒	🕒
Analytics / reports	✓	✓	🕒	🕒	✕	✕
Moderate reviews	✓	✓	✕	✕	✕	✕
Submit review	✕	✕	✕	✕	✕	🕒
Run AI agents	✓	✓	🕒	🕒	🕒	🕒
Read audit log	✓	✓	✕	✕	✕	✕

RLS mechanism: scoping columns (`tenant_id` , `owner_doctor_id` , `clinic_id` , `assigned_manager_id` , `patient_id`) are compared against `auth.uid()` and the user's role row in every policy. The AI service role uses scoped service credentials and writes an `audit_log` entry per action.

4.1 Multi-tenancy — a clinic/hospital is a tenant

The platform is **multi-tenant (B2B2C)**: each clinic/hospital is a **tenant** whose staff and data are isolated from every other tenant. The Super Admin (platform) sits above all tenants; Admins/Managers/Doctors/Front-desk live inside one tenant; Patients are cross-tenant consumers scoped to their own records.

- **Every table carries `tenant_id`** ; **every** query and RLS policy is tenant-scoped — tenant isolation is mandatory, no shared data unless explicit.
- Tenant context is validated server-side on every request; tenant access is audited.
- Isolation is tested regularly; security before convenience. Shared infrastructure is fine, shared *data* is forbidden.

5. Core Feature Modules

5.1 Doctor onboarding & listing

Sign-up → build profile (name, specialties, qualifications, experience, languages, fees, clinic & location, hours, photos) → upload proof for **verification/KYC** → Admin approves → listing goes live (SEO page). Doctors manage their own staff and availability.

5.2 Discovery & search

Browse by specialty/area; filter by specialty, location, availability, gender, language, fee, rating. **AI Smart Search** turns free text ("child fever doctor near Fatehsagar, evening") into ranked matches using hybrid keyword + vector search and business rules.

5.3 Lead capture (the popup)

- Triggered by **Call** / **Book** on any profile or search card.
- Collects name, **phone (OTP-verified)**, problem/symptoms, preferred time, consent.
- **AI Lead-Form Assistant** validates & normalizes input, summarizes the problem, de-dupes against existing leads, and tags the likely specialty.
- Creates a `Lead` with **source tracking** (which doctor/page/campaign).
- Sends instant SMS/WhatsApp confirmation; raises a realtime alert to staff.

5.4 Lead pipeline

Kanban/list of leads with status **New** → **Contacted** → **Booked** → **Closed/Lost**, assignment, notes, an **SLA timer** (time-to-first-contact), and full activity history (`Lead_events`).

5.5 Appointment booking

Per-doctor **slot calendar** from availability rules; create/confirm/reschedule/cancel; double-booking prevention; automatic **reminders**; no-show tracking.

5.6 Dashboards & analytics

Role-scoped dashboards: lead funnel, conversion, SLA breaches, bookings, show-rate, top doctors/specialties, source attribution. Live via Supabase Realtime.

5.7 Notifications

SMS / WhatsApp / email for lead & booking confirmations and reminders, behind a provider adapter (MSG91 / Gupshup / Twilio) so we can swap vendors.

5.8 Reviews & ratings (P2)

Post-visit rating requests; moderation queue; verified-visit badge; feeds search ranking.

6. The AI Layer — Multiple Agents & LLM Choice

Everything AI runs through the **Vercel AI Gateway** (one API, provider fallbacks, observability, optional zero-data-retention — important for health data) built with the **AI SDK**. We **tier models by task** to keep cost low.

6.1 Which LLM to use (direct recommendation)

Tier	Model	Use it for	Why
Cheap / high-volume	Claude Haiku 4.5	lead parsing & validation, classification, summaries, first-pass triage	fast & low-cost at scale
Reasoning (default)	Claude Sonnet 4.6	smart-search synthesis, staff copilot, content generation	best quality-per-rupee for real reasoning
Hardest only	Claude Opus 4.8	complex multi-step agentic flows, long-context analysis	top capability, used sparingly
Embeddings	small embedding model (text-embedding-3-small or open BGE)	semantic vectors for doctor search	cheap; stored in Supabase pgvector

Bottom line: default to **Claude** via the AI Gateway, start most calls on **Haiku**, escalate to **Sonnet** when quality needs it, reserve **Opus** for the few hard cases. This single decision is the biggest lever on your AI bill.

6.2 The five agents

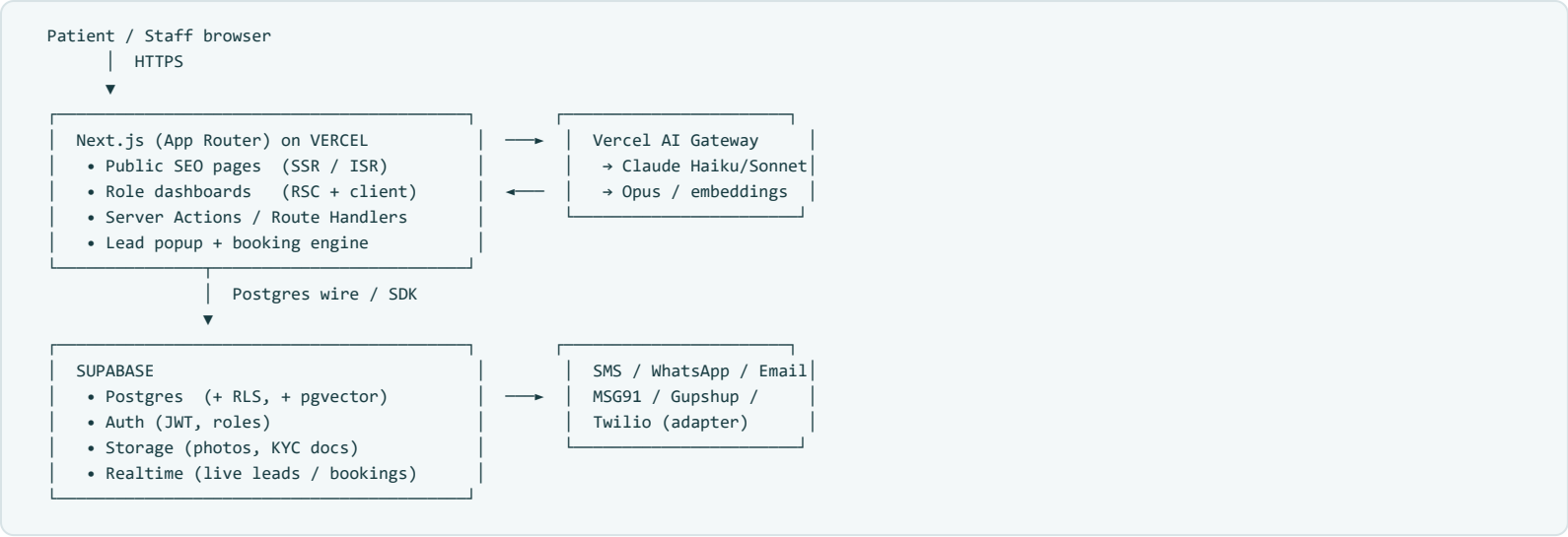
#	Agent	Job	Model	Phase
1	Smart Search & Match	free text → ranked doctors (vector + keyword + rules)	Sonnet + embeddings	1
2	Lead-Form Assistant	autofill, validate, transcribe, summarize, de-dupe leads	Haiku	1
3	Symptom Triage	symptoms → suggested specialty (<i>triage, not diagnosis</i>)	Haiku→Sonnet	2
4	Staff Copilot	lead summaries, suggested replies, follow-up nudges, daily reports	Sonnet	2
5	Content & SEO	doctor bios, specialty pages, local SEO copy	Sonnet	2

6.3 Guardrails (all agents)

- Run under the **AI service role**; **every action audit-logged** (`ai_runs`).
- **PII redaction** before model calls where possible; honour DPDP consent.
- Token + rate limits and a **cost ceiling per tenant**.
- **Tools are RLS-scoped** — an agent only sees what its caller's role can see.
- **Medical safety:** triage shows a clear disclaimer, never diagnoses, and routes red-flag symptoms (chest pain, stroke signs, breathing difficulty) straight to emergency guidance.

Compliance note: the symptom agent is a *triage helper*, not a medical device. Keep the disclaimer visible and log consent.

7. System Architecture



Component responsibilities

Layer	Tech	Notes
Frontend	Next.js App Router on Vercel	ISR for SEO doctor/specialty pages; RSC dashboards; mobile-first.
Backend	Server Actions + Route Handlers (Fluid Compute)	full Node.js, low cold-starts; webhooks & AI endpoints here.
Database	Supabase Postgres	RLS on every table ; pgvector for embeddings; Realtime for dashboards.
Auth / RBAC	Supabase Auth	role in <code>user_roles</code> ; enforced by RLS + server checks.
AI	Vercel AI Gateway + AI SDK	agents in <code>/agents</code> ; tiered Claude models.
Messaging	Provider adapter	swap MSG91 / Gupshup / Twilio without code churn.

Why this stack: minimal ops, generous free tiers, scales to millions, and the managed primitives (Auth, RLS, Storage, Realtime, AI Gateway) mean we write product code, not plumbing — ideal for an agent-built project.

8. Data Model

Conventions: snake_case; every table has `id` `uuid` `pk` , `tenant_id` (**clinic**), `created_at` , `updated_at` ; money in **paise (int)**; timestamps UTC (shown `Asia/Kolkata`); phones **E.164. RLS on every table is tenant-scoped** — the database is the single source of truth; all schema changes ship as migrations (no manual prod edits).

Core tables

Table	Purpose	Key columns
profiles	one row per auth user	id, full_name, phone, locale
user_roles	role assignment	user_id, role, scope (clinic/territory)
clinics	physical clinics	id, name, address, geo, owner_id
doctors	doctor entity	id, profile_id, clinic_id, exp, fee_paise, verified
specialties	master list	id, name, slug
doctor_specialties	M:N link	doctor_id, specialty_id
listings	public listing content	doctor_id, bio, photos, status (draft/live)
availability_slots	bookable time	doctor_id, start, end, status
leads	captured enquiries	patient_id, owner_doctor_id, clinic_id, assigned_manager_id, status, source, ai_summary
lead_events	lead activity log	lead_id, actor, type, note, at
bookings	confirmed appts	lead_id, doctor_id, slot_id, patient_id, status
reviews	ratings	doctor_id, patient_id, rating, text, moderated
notifications	outbound messages	to, channel, template, status
doctor_embeddings	semantic search	doctor_id, embedding vector
ai_runs	AI audit	agent, role, input_hash, tokens, cost, at
audit_log	security audit	actor, action, entity, before/after, at

Relationships: a clinic has many doctors; a doctor has many listings, slots, leads, bookings, reviews; a lead may convert to one booking; every lead/booking carries the scoping ids used by RLS.
Reserved-but-unused tables for later: `payments` , `consultations` , `prescriptions` , `lab_orders` .

9. Frontend Specification

Page inventory

Area	Pages
Public (SEO, ISR)	Home · Specialty landing · Area landing · Search results · Doctor profile · About/Contact
Conversion	Lead popup modal · Booking modal · OTP verify · Confirmation
Patient area	My bookings · Lead status · Profile
Staff dashboards	Super Admin · Admin · Manager · Doctor · Front-desk (role-gated)
Doctor tools	Listing editor · Availability/slots · Lead inbox · Calendar · Reports

Design system (lightweight)

Tone: clean, trustworthy, calm medical palette (teal/white), large tap targets, fast on low-end Android.

Build: Tailwind + shadcn/ui components, RSC by default, client islands only where needed (popup, calendar, dashboards).

- Typography: one humanist sans; clear hierarchy; min 16px body on mobile.
- Accessibility: labelled forms, focus states, color-contrast AA, keyboard-navigable modal.
- Bilingual-ready copy (Hindi + English) via i18n keys from day one.

10. Backend & API Specification

Mutations use **Server Actions**; webhooks and AI use **Route Handlers**. RBAC is enforced by RLS *and* a server-side check in each action.

Surface	Examples	Auth
Lead intake	<code>submitLead()</code> , <code>verifyOtp()</code>	public + OTP
Search / AI	<code>POST /api/search</code> , <code>POST /api/triage</code>	public (rate-limited)
Lead ops	<code>assignLead()</code> , <code>updateLeadStatus()</code>	Manager/Admin (RLS)
Booking	<code>createBooking()</code> , <code>reschedule()</code>	staff / patient (own)
Listing	<code>upsertListing()</code> , <code>verifyDoctor()</code>	Doctor / Admin
Webhooks	<code>/api/webhooks/sms</code> , <code>/api/webhooks/whatsapp</code>	signed
Cron	reminder dispatch, SLA escalation, daily digest	Vercel Cron

Pattern: every action → (1) authenticate, (2) authorize via role + RLS, (3) validate input (zod), (4) mutate, (5) write `audit_log` / `lead_events` , (6) fire notification/realtime event.

11. Deployment — Vercel + Supabase

Pipeline

```
git push branch → Vercel Preview deploy (unique URL) → review/QA
└→ merge to main → Vercel Production deploy → promote
                        Supabase migrations applied via CI / Supabase MCP (scoped write)
rollback = re-promote previous Vercel build (+ down-migration if schema changed)
```

Environments

- **Preview** (per-branch) and **Production** on Vercel.
- Supabase: separate project (or branch DB) per environment.
- Migrations live in `supabase/migrations` ; never edit prod schema by hand.

Environment variables (inventory)

NEXT_PUBLIC_SUPABASE_URL	SUPABASE_SERVICE_ROLE_KEY
NEXT_PUBLIC_SUPABASE_ANON_KEY	AI_GATEWAY_API_KEY
SUPABASE_PROJECT_REF	SMS_PROVIDER_KEY / WHATSAPP_TOKEN
SUPABASE_ACCESS_TOKEN (MCP)	OTP_SECRET / CRON_SECRET

Secrets rule: never commit secrets. They live only in Vercel/Supabase env settings. `.mcp.json` references `${VARS}` only.

12. Cost Analysis (INR + USD)

Estimates as of mid-2026; confirm live pricing on each dashboard. USD≈₹85. AI cost depends on volume & model tiering — the ranges assume aggressive use of Haiku.

Per-service pricing

Service	Free / entry	Paid tier	What you get
Vercel	Hobby — \$0	Pro — ~\$20/mo (₹1,700)	builds, hosting, functions, cron, preview deploys
Supabase	Free — \$0	Pro — ~\$25/mo (₹2,100)	Postgres, Auth, Storage, RLS, pgvector, Realtime
AI (Claude via Gateway)	pay-per-token	scales with usage	Haiku cheap · Sonnet mid · Opus premium
SMS	—	~₹0.15–0.25 / SMS	OTP & confirmations
WhatsApp	—	~₹0.10–0.35 / conversation	confirmations & reminders
Domain	—	~₹900 / year	.in / .com

Monthly total by stage

Stage	Assumptions	Vercel	Supabase	AI	SMS/WA	Total / mo
MVP (Udaipur)	~1k leads, ~10k searches	Free–Pro	Free	₹500–2,000	₹500–1,500	₹2,000–5,000
Growth	~10k leads, ~100k searches	₹1,700	₹2,100	₹5,000–15,000	₹3,000–8,000	₹15,000–30,000
Scale	~100k leads, multi-city	₹1,700+	₹8,000+	₹30,000–90,000	₹20,000–60,000	₹60,000–1,60,000

Stay near-free in MVP: Vercel Hobby + Supabase Free + Haiku for almost everything + SMS only for OTP (WhatsApp for the rest). Realistic early bill: a few hundred to a couple thousand rupees a month.

Cost-control levers

- **Model tiering** (Haiku first) — the single biggest lever.
- **Cache** embeddings & frequent AI answers; reuse doctor vectors.
- **Rate-limit** public AI endpoints; cap tokens per call; per-tenant ceilings.
- **Prefer WhatsApp** over SMS where possible; SMS only for OTP.
- **ISR caching** cuts function invocations on SEO pages.

13. Security & Compliance (India / DPDP)

Area	Approach
DPDP Act 2023	explicit consent at lead capture; purpose limitation; data-principal rights (access/erase); consent + retention logged.
Health-data sensitivity	minimize collection; symptom text treated as sensitive; encrypt at rest (Supabase) & in transit (HTTPS).
Authorization	least-privilege RLS on every table; server-side checks; no client-trusted authorization; frontend visibility is never security.
Multi-tenant isolation	every query & policy scoped by <code>tenant_id</code> ; tenant context validated server-side; no cross-tenant leakage; isolation tested regularly.
Auditability	<code>audit_log</code> for staff actions, <code>ai_runs</code> for AI actions, <code>lead_events</code> for lead history.
AI safety	medical disclaimers; triage-not-diagnosis; red-flag routing; PII redaction before model calls; zero-retention gateway option.
Secrets	env-var only; scoped tokens; rotation; read-only MCP by default.
Future: ABDM/ABHA	data model leaves room for ABHA health-ID linkage and consent-manager flows when we add records/teleconsult.

Not a medical device. The platform assists discovery and booking; it does not provide diagnosis or treatment. All AI medical output carries a disclaimer.

14. Build Methodology — Claude Code + Codex + MCP

This is the part you emphasized: **how** the project is built so nothing is forgotten and nothing is overwritten blindly. Two AI coding agents collaborate through markdown files, a shared MCP toolchain, and a strict version-backup discipline.

14.1 Two-agent division of labour your "Code vs Codex" question

Claude Code — <i>architect & reviewer</i>	Codex — <i>builder</i>
Architecture, data model, Supabase schema + RLS + migrations	UI components, forms, dashboards, list/table screens
The AI layer (agents, prompts, tools, gateway wiring, embeddings)	Tests, fixtures, seed data
Security/compliance, auth/RBAC, audit logging	Boilerplate CRUD & route wiring once schema is fixed
Code review of Codex output; tricky bugs; final integration	Repetitive, well-specified work matching existing patterns

Each agent reads its own instruction file: `CLAUDE.md` (Claude Code) and `AGENTS.md` (Codex). They exchange work via `docs/HANDOVER.md`.

14.2 Repository file & folder structure your "what files" question

f:\Doctor\ ├ README.md ├ CLAUDE.md ├ AGENTS.md ├ .mcp.json ├ Doctor_Platform_Blueprint.pdf ├ docs/ │ ├── PRD.md │ ├── ARCHITECTURE.md │ ├── RBAC.md │ ├── AI-AGENTS.md │ ├── MCP.md │ ├── HANDOVER.md │ ├── DECISION_LOG.md │ └ VERSIONING.md ├ .versions/ ├ app/ ├ components/ ├ lib/ ├ agents/ └ supabase/	 project overview + rules (entry point) instructions for Claude Code (the architect) instructions for Codex (the builder) shared MCP servers (supabase, vercel, playwright, fs) this master spec product requirements system + data architecture roles + permission matrix the 5 agents + LLM choice MCP setup & policy live hand-off between the two agents dated, append-only record of every decision backup-before-new-version discipline frozen snapshots: v1/ v2/ ... (never edited) Next.js routes (created during build) UI components utilities, supabase client, auth helpers AI agent definitions, prompts, tools schema + migrations + seed
--	--

14.3 The markdown memory system "save every chat & contact"

- `docs/DECISION_LOG.md` — append-only; every decision with date, what, why, who. Never delete an entry. This is the project's long-term memory.
- `docs/HANDOVER.md` — the living "current state": what was just done, what's next, open questions. Overwrite the top each session; append to History.
- Contacts & key facts** — captured in the relevant doc (e.g. provider contacts in `ARCHITECTURE.md`), and important cross-session facts saved to the assistant's memory so they survive new chats.

14.4 Versioning & backup "keep old version before new"

Before starting any new version, **snapshot the current working tree into** `.versions/vN/` (the version you are leaving), tag the git commit, and add a row to the version table in `VERSIONING.md`. Snapshots are read-only and never deleted.

before building v2, freeze v1: \$v = "v1" New-Item -ItemType Directory -Force ".versions/\$v" Out-Null robocopy . ".versions/\$v" /E /XD .git node_modules .versions /NFL /NDL Out-Null git tag v1
--

14.5 "Empty / placeholder files" explained your "empty files" question

We create some files **before** there is code in them, on purpose:

Placeholder	Why it exists empty/early
<code>docs/HANDOVER.md</code>	so either agent always has a known place to read/write status
<code>docs/DECISION_LOG.md</code>	so decisions are logged from commit #1
<code>.versions/</code> (with README)	reserves the backup location before the first snapshot

<code>.mcp.json</code>	defines the shared toolchain before any feature code
<code>agents/</code> , <code>supabase/</code> stubs	fix the structure so both agents put things in the same place (consistency)

14.6 MCP toolchain "make MCP"

A shared `.mcp.json` gives both agents the same safe tools (full policy in `docs/MCP.md`):

Server	Use	Policy
supabase	inspect schema, queries, migrations, types	read-only by default; scoped write only for a deliberate migration session
vercel	deployments, logs, env, status	hosted HTTP MCP (OAuth)
playwright	E2E checks, screenshots, render PDFs	used to verify UI
filesystem	scoped file access	locked to <code>f:\Doctor</code>

Future product MCP: we may expose our own MCP server so internal agents can query leads/bookings (RLS-scoped, audit-logged) — designed around the AI service role.

Consistency rule: both agents use the *same* `.mcp.json` , the *same* naming conventions, and the *same* folder structure. Any new server or convention is recorded in `DECISION_LOG.md` first.

15. Engineering Operating System — The 8-Agent Roster

The platform is built by **eight specialized agents**, orchestrated across **Claude Code** (architecture, AI, review — the "thinking" roles) and **Codex** (high-throughput building). Each agent has one job, consumes a defined input, and produces a defined output that the next agent depends on. **AI never becomes the source of truth** — a human override always exists.

Agent	Mission	Consumes	Produces	Runs as
PM Agent	turn needs into scoped requirements	user goals, market research	PRD.md , acceptance criteria, task breakdown	Claude Code
Architect Agent	design clean, modular architecture	PRD	architecture.md , ADRs, module boundaries	Claude Code
Database Agent	schema, RLS, indexes, integrity	architecture	migrations, RLS policies, ER docs	Claude Code
Backend Agent	APIs & business logic	schema + ADRs	server actions, route handlers, validation	Codex → CC review
Frontend Agent	UI & user flows	API + designs	Next.js pages, Shadcn components, states	Codex
AI Agent	the 5 product agents + safeguards	data + APIs	prompts, tools, gateway wiring, guardrails	Claude Code
QA Agent	verify behaviour, security, RLS	built feature	tests, RLS/permission checks, regression suite	gate (CC + Codex)
Documentation Agent	keep docs synced with reality	completed task	PRD/CHANGELOG/USER_FLOWS/API + handover notes	gate

Hand-off chain

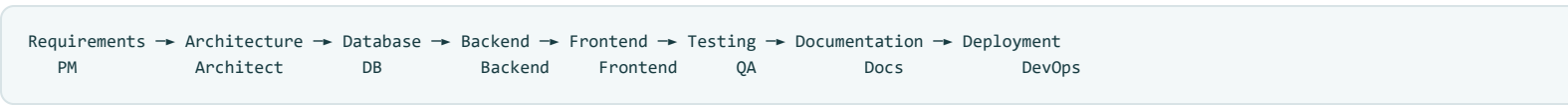
PM → Architect → Database → Backend → Frontend → QA → Documentation → Deploy

every arrow = a handover note (docs/HANDOVER.md) + a decision entry if anything was decided

Orchestration: Claude Code owns PM/Architect/Database/AI and reviews everything; Codex owns Backend/Frontend build; QA & Documentation are **gates** — a task is not "done" until both pass. Mapping detail lives in docs/AGENT-ROSTER.md .

16. Delivery Workflow & Traceability

Every feature flows through one pipeline. **Documentation, backups and handovers are never skipped.** Work proceeds in **small modular steps** — Plan → Build → Test → Log → Repeat.



Every feature must update (no exceptions)

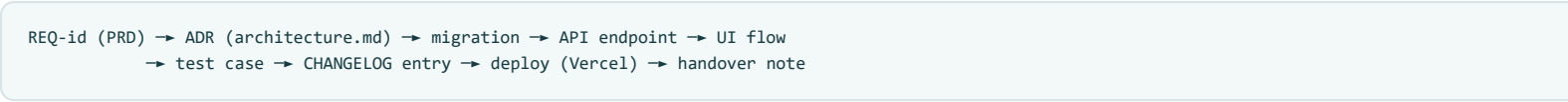
Docs

- `PRD.md` — the requirement
- `CHANGELOG.md` — what changed
- `USER_FLOWS.md` — affected flows
- API docs — endpoints & contracts
- `dev-log.md` , `tasks.md` — progress

Code & safety

- migration file for any schema change
- tests with every feature (QA gate)
- backup/snapshot before major change
- handover note after every task
- `DECISION_LOG.md` if a decision was made

Traceability — requirement → deployment



Any shipped line of code can be traced back to a requirement, and any requirement traced forward to its deployment. **Correctness before speed; security before features.**

Hard gates: no release without tests passing · no schema change without a migration · no decision overwrite without a `DECISION_LOG.md` update · no major change without a `.versions/` backup.

17. Documentation System & Skills

Documentation is treated as a first-class deliverable, kept **synchronized with the implementation** at all times. Every required file has a single, clear purpose.

File	Holds	Updated by
PRD.md	product requirements, acceptance criteria	PM
architecture.md	tech decisions, libraries, rendering, perf strategy	Architect
tasks.md	pending / completed / next-priority tasks	all
dev-log.md	what was built, what changed, issues + fixes	all
CHANGELOG.md	versioned list of changes	Documentation
USER_FLOWS.md	step-by-step user journeys	Documentation
API.md	endpoints, contracts, error formats	Backend/Docs
DECISION_LOG.md	append-only decisions (never overwrite)	all
HANDOVER.md	live hand-off between agents/sessions	all
GOVERNANCE.md	the full rulebook (Ch. 18 mirror)	Architect

Skills-first rule

- **Before building, check the local /skills directory.** If a relevant skill exists (e.g. Supabase/Postgres best practices, Shadcn UI, frontend-design), **reuse it**.
- If a needed skill is missing, install/download it before proceeding — don't reinvent.
- Persist progress continuously in dev-log.md / architecture.md / tasks.md so any agent can resume with zero context loss.

Backups & consistency: snapshot to .versions/vN before major changes; keep naming, structure and conventions uniform across every file.

18. Governance Rulebook (full)

The complete operating ruleset for this and future projects. Mirrored to `/docs/GOVERNANCE.md` for the agents. Condensed for print; each line is a rule.

Core Principles

Security before features · correctness before speed · simplicity before complexity · scalability before shortcuts · maintainability before hacks · reuse before duplication · reliability before optimization · automation before manual · UX before visual effects · **data integrity is non-negotiable**.

Code Quality

No duplication · no dead code · no unused deps · no magic values · meaningful names · small focused functions · single responsibility · maintainable files · readability over cleverness · every change improves the codebase.

Database

DB is source of truth · migrations only · no manual prod edits · timestamps on every record · audit logs on critical actions · never trust client data · proper indexes · optimized queries · protect integrity · design for growth.

Security

Deny by default · least privilege · encrypt sensitive data · protect secrets · never expose keys · server-side authz only · audit privileged actions · monitor anomalies · MFA where applicable · security review before release.

Authorization

Backend enforces perms · FE visibility ≠ security · role checks on every action · resource- & tenant-level perms · admin actions logged · sensitive actions confirmed · no implicit trust · least privilege.

Frontend

Fast loading · responsive · accessible · consistent reusable components · proper loading/error states · safe optimistic updates · offline when required · UX first.

AI

Never the source of truth · output verifiable · human override always · log decisions, prompts & outputs · protect user data · avoid hallucination · validate responses · safeguards on actions · fail safely.

Realtime

DB remains truth · realtime is delivery only · handle reconnects & conflicts · idempotent updates · tenant-scoped channels · monitor latency · recover gracefully · sync after reconnect.

Performance

Measure before optimizing · cache intelligently · lazy-load heavy content · minimize DB load · optimize requests & assets · monitor continuously · reduce bundle size · CDN · user-perceived speed first.

DevOps

Infra as code · automated deploys & backups · monitoring + alerts · disaster recovery · rollback capability · environment separation · secret management · operational visibility.

Product

Solve real problems · user value · avoid bloat · measure adoption & outcomes · collect feedback · iterate · prioritize reliability & retention · build for long-term sustainability.

Architecture

Clean architecture · separate UI / logic / data · no business logic in UI · loosely coupled, highly cohesive · design for scale · domain-driven · event-driven where useful · avoid premature microservices · start modular, scale when needed.

TypeScript

Strict mode · never `any` · explicit types · consistent interfaces/types · validate external data · no unsafe casts · shared FE/BE types · strong typing for APIs, DB models, everywhere.

API

Validate every request & response · never trust FE input · consistent response format · proper error handling · authn + authz · rate-limit sensitive endpoints · log failures · version breaking APIs.

Authentication

Trusted providers (Supabase Auth) · no manual password storage · secure sessions · role-based perms · session expiry · audit auth events · prevent privilege escalation · secure reset · verify identity for sensitive actions.

Multi-tenancy

Tenant isolation mandatory · no data leakage · every query scoped · tenant context validated · audit tenant access · per-tenant perms · shared infra ok, shared data forbidden unless explicit · isolation tested regularly.

Backend

Stateless where possible · centralized business logic · reliable error handling · structured logging · idempotent ops · background jobs for heavy work · secure endpoints · scalable services · observability.

File Storage

Object storage · metadata separate · validate uploads · restrict access · signed URLs · encrypt sensitive files · scan when possible · CDN delivery · backup critical assets · lifecycle management.

Offline

Queue writes · auto-sync · safe conflict resolution · show sync status · protect integrity · visible failures · minimal local data · encrypt local sensitive data · audit sync.

Testing

Test critical workflows, business logic, security, permissions, integrations, edge cases & failures · automate · prevent regressions · **no release without validation**.

Compliance

Minimize data · protect PII · audit trails · appropriate retention · support deletion · track consent · encrypt sensitive info · follow regional regs (DPDP) · log compliance actions · regular reviews.

Golden Rules

Security first · data is sacred · users are the priority · simplicity wins · reliability · scalability · maintainability · observability · automation · **long-term thinking beats short-term shortcuts**.

19. Phased Project Plan

Phase	Milestones	AI agents	Rough effort
0 — Setup	repo + docs scaffold (done), Next.js + Supabase init, env + MCP, CI/CD, base schema + RLS	—	~1 week
1 — MVP	listings + onboarding, search, lead popup , lead pipeline, booking, RBAC dashboards, notifications (OTP+confirm)	Smart Search, Lead-Form Assistant	~4–6 weeks
2 — Engage	reminders, reviews, analytics, symptom triage, staff copilot, content/SEO, more cities	+ Triage, Staff Copilot, Content/SEO	~4–6 weeks
3 — Expand	payments, teleconsult, labs/medicine, health records, ABDM/ABHA	agent suite extended	roadmap

Definition of done (every feature)

Builds clean · RLS verified per role · decision logged · handover updated · version snapshotted on a version bump · tests pass.

20. Appendices — Ready-to-paste Templates

These also exist as live files in the repo. Reproduced here for handover.

A. AGENTS.md (Codex) — skeleton

```
# AGENTS.md — Instructions for Codex
Project: AI-first doctor listing + lead + booking portal (Next.js + Supabase on Vercel).
You build; Claude Code architects (see CLAUDE.md). Read docs/HANDOVER.md for current task.
OWN: UI, forms, dashboards, tests, CRUD wiring (after schema fixed).
HAND BACK: schema/RLS, AI agents, auth/RBAC, compliance, new deps.
RULES: match existing patterns · log to DECISION_LOG.md · update HANDOVER.md · snapshot before
       version bump · no secrets in code · use .mcp.json servers.
```

B. Handover entry — template

```
## Current state
Date: YYYY-MM-DD · From:  · To:  · Version: vN
What was done: ...
What's next: 1) ... 2) ...
Open questions: ...
Don't forget: snapshot before version bump; log decisions.
```

C. Decision-log entry — template

```
### YYYY-MM-DD —
- Decision: ...
- Why: ...
- By:
```

D. Starter SQL (shape only)

```
create table doctors (
  id uuid primary key default gen_random_uuid(),
  profile_id uuid references profiles(id),
  clinic_id uuid references clinics(id),
  fee_paise int not null default 0,
  verified boolean not null default false,
  created_at timestampz default now(),
  updated_at timestampz default now()
);
alter table doctors enable row level security;
-- policy: a doctor can read/update only their own row; admins all.
```

E. Glossary

Term	Meaning
Lead	a captured patient enquiry (from the popup) before it becomes a booking
RLS	Row Level Security — Postgres row-level authorization
RBAC	Role-Based Access Control
ISR	Incremental Static Regeneration — cached, SEO-friendly pages
MCP	Model Context Protocol — safe tool access for AI agents
pgvector	Postgres extension storing embeddings for semantic search
DPDP	India's Digital Personal Data Protection Act, 2023
ABDM/ABHA	India's health-ID / digital health mission (future)

You are ready to build. Start at `README.md`, follow `CLAUDE.md` / `AGENTS.md`, keep `HANDOVER.md` and `DECISION_LOG.md` alive, snapshot before every new version, and let the five agents do the heavy lifting.